

FRAGMENT: Factorized Graph Representations for Document Generation and Editing via Entity-Aware Transformations

Ayoub El Bouchtili
IBM Consulting

ayoub.elbouchtili@fr.ibm.com

Guillaume Leroy-Meline
IBM Consulting

guillaume@fr.ibm.com

Abstract. Structured documents such as invoices, forms, reports, and scientific articles derive meaning from the interplay between spatial layout, textual content, and logical structure. Generative models operating at the pixel or token level often struggle to capture these dependencies effectively. We explore FRAGMENT, a generative framework that represents a document as a typed relational graph and factorizes its distribution as

$$p(\text{structure}, \text{content}) = p(\text{structure}) \cdot p(\text{content} \mid \text{structure}).$$

The framework consists of two stages. The first stage, the *Architect*, is a causally masked Transformer conditioned on the document category that autoregressively generates the document graph, including node types, typed spatial relations, and graph connectivity through a pointer-based mechanism. The second stage, the *Builder*, is a GATv2-based graph attention network with residual pre-normalized message passing that enriches the generated graph with normalized bounding boxes, textual content, and visual attributes such as font size, font family, and color through dedicated prediction heads.

Both stages define explicit likelihood models. The Architect models structural decisions through categorical distributions, while the Builder employs attribute-specific observation models for layout, text, and style. This factorized formulation yields a tractable document-level likelihood that serves as an anomaly score for synthetic and manipulated document detection. To support controlled document editing, we further introduce a prompt-conditioned extension that injects instruction embeddings into the Builder through cross-attention, enabling semantic and entity-aware modifications.

We describe the training methodology on DocLayNet and the fine-tuning procedures for FUNSD and SROIE. We also outline an evaluation protocol covering layout fidelity, semantic quality, and forgery detection. Experiments on DocLayNet, FUNSD, and SROIE evaluate FRAGMENT alongside representative autoregressive, layout-only, and graph-based baselines, providing an empirical analysis of the characteristics and trade-offs of the proposed factorized graph generation framework.

1 Introduction

Structured documents such as invoices, forms, reports, and scientific articles convey information through the joint organization of layout, textual content, and logical structure. Unlike natural images or free-form text, their meaning depends not only on the presence of individual elements, but also on their spatial arrangement and semantic relationships. A table caption derives meaning from its association with a table, a total is linked to the line items that precede it, and a signature field occupies a specific role within a form. Generative models that operate directly on pixels or token sequences must implicitly recover these dependencies, which can lead to structural inconsistencies across the page.

In this work, we explore representing documents as typed relational graphs. In this representation, nodes correspond to semantic layout elements, edges capture spatial and logical relationships, and node attributes encode geometry, text, and visual style. Such a representation exposes the hierarchical organization of a document and separates structural planning from content realization.

Based on this perspective, we present FRAGMENT (Factorized Representation of Artifacts via Graph-based Modeling and Entity-aware Transformations), an exploratory graph-based generative framework for structured documents. Rather than generating a document as a single sequence of pixels or tokens, FRAGMENT separates generation into two complementary stages. The *Architect* first generates a typed relational scaffold describing the semantic entities and their relations. The *Builder* then realizes this scaffold by predicting geometric, textual, and visual attributes. This separation

enables explicit modeling of document structure while preserving the flexibility required for content generation. The *Architect* produces a document graph that defines the structural scaffold, including semantic element types and their relationships. The *Builder* then instantiates this scaffold by predicting layout attributes, textual content, and visual styling. This separation mirrors a common document creation workflow, where a structural plan is established before individual content elements are populated.

The resulting framework offers two main design properties. First, because each generation stage defines an explicit likelihood model, FRAGMENT assigns a decomposable document-level likelihood whose components correspond to structure, layout, text, and style. This property enables AI-forgery detection, where anomalous documents can be analyzed through their likelihood under the learned generative process (Section 7). Second, the explicit graph structure provides a natural scaffold for targeted, instruction-based editing (Section 6).

Our contributions are as follows:

1. We formulate a unified graph representation for structured documents together with an R-tree-based construction procedure that extracts typed spatial relations from layout annotations (Section 3).
2. We propose a two-stage generative architecture that separates structural generation from content realization through an autoregressive graph-based Architect and a GATv2-based Builder (Sections 4–5).
3. We define a composite document log-likelihood that provides a measure of document plausibility for forgery detection (Section 7).
4. We introduce a prompt-conditioned editing mechanism based on cross-attention that supports semantic and entity-aware document modification (Section 6).
5. We present the training methodology and an evaluation protocol examining layout fidelity, semantic quality, and forgery detection across benchmark datasets (Sections 8).

2 Related Work

Layout generation. Early neural approaches generated page layouts with adversarial training (LayoutGAN [5]) or variational Transformers [6]. LayoutTransformer [7] showed that self-attention over discretized element sequences is a strong autoregressive baseline, and LayoutDM [8] brought discrete diffusion to controllable layout generation. These methods generate *geometry only*; text and style must be supplied externally. FRAGMENT instead explores treating layout as one attribute of a broader document graph whose structure, content, and style are generated jointly under a factorized likelihood.

Graph-based document modeling. Representing documents as graphs has proven effective for both analysis and synthesis. Biswas et al. [9] generate document layouts with graph neural networks; AliGATr [10] generates form layouts with graph attention, and graph-based synthetic layouts have been used to augment Document AI training data [11]. Autoregressive graph generation itself traces back to GraphRNN [12]. FRAGMENT follows this lineage by coupling an autoregressive *structure* generator with a message-passing *content* generator, using the graph as the structural backbone of the generative process.

Document AI and multimodal document models. Recent large multimodal models have further advanced document understanding by jointly reasoning over text, images, and layout. However, these approaches generally operate through implicit representations and do not provide an explicit document structure or tractable generation likelihood. In contrast, FRAGMENT models documents as structured graphs, providing explicit intermediate representations for generation, editing, and likelihood-based evaluation.

Discriminative document understanding has been transformed by layout-aware pre-training: LayoutLM [13] and LayoutLMv3 [14] fuse text, layout, and image; UDOP [15] unifies the three modalities in one encoder-decoder; DocLLM [16] extends large language models with spatial attention. On the generative side, DocSynthv2 [17] autoregressively generates layout and text jointly in a single flat sequence. FRAGMENT explores making the structural scaffold explicit and generating it first, enabling category-controllable structure and a decomposable likelihood.

Likelihood-based anomaly and forgery detection. Using generative model likelihoods to flag out-of-distribution inputs is attractive but subtle: raw likelihoods can be miscalibrated across domains [18], motivating corrections such as typicality tests [19] and likelihood ratios [20]. Document forgery detection is an active field with dedicated benchmarks emerging for AI-manipulated documents [2, 3]. FRAGMENT investigates a *structure-aware* likelihood score: because the total log-likelihood decomposes over graph components and attribute heads, potential anomalies can be localized (e.g., “this total field’s text is unlikely given its neighborhood”), providing fine-grained likelihood feedback.

3 Document Graphs

FRAGMENT represents document generation as a factorized probabilistic process:

$$p(\text{structure}, \text{content}) = p(\text{structure}) \cdot p(\text{content} \mid \text{structure}). \quad (1)$$

The first term corresponds to the generation of the document scaffold, including semantic entities and their relations, while the second term models the realization of this scaffold through geometry, text, and visual attributes. In practice, the structural distribution is parameterized by the *Architect*, whereas the conditional realization distribution is parameterized by the *Builder*.

Documents are represented as typed attributed graphs $G = (V, E, \tau, \rho, A)$, where each node $v \in V$ corresponds to a semantic layout element with type $\tau(v) \in \mathcal{T}$ (e.g., title, text, table, picture; $|\mathcal{T}| = 11$ for DocLayNet), each directed edge $e \in E$ carries a relation type $\rho(e) \in \{\text{BELOW}, \text{RIGHT-OF}\}$, and A collects node attributes: a normalized bounding box $\mathbf{b}_v \in [0, 1]^4$ in (x_1, y_1, x_2, y_2) form, a token sequence $\mathbf{t}_v$ (WordPiece, max length 50), and style attributes (s_v, f_v, c_v) for font size (normalized to $[0, 1]$), font family (categorical over a normalized font vocabulary), and RGBA color. Every graph is additionally labeled with a document category d drawn from the DocLayNet taxonomy (financial reports, government tenders, laws and regulations, manuals, patents, scientific articles).

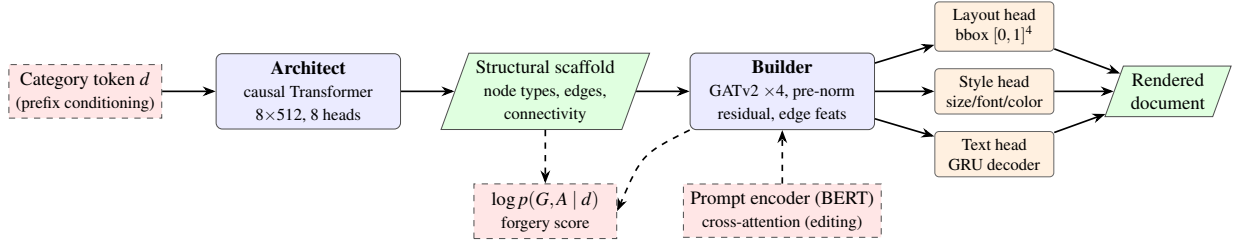


Figure 1: FRAGMENT pipeline. The Architect autoregressively generates a typed document graph conditioned on a category prefix; the Builder contextualizes the scaffold with GATv2 message passing and realizes layout, style, and text through dedicated heads. Dashed paths show the two auxiliary capabilities: prompt-conditioned editing via cross-attention, and the composite log-likelihood used for forgery detection.

Graph construction. Given layout annotations (or OCR output), we build edges with an R-tree spatial index for computational efficiency. A directed BELOW edge $u \rightarrow v$ is created when v lies below u and their horizontal projections overlap by more than 40% of u ’s width; RIGHT-OF edges are defined symmetrically with a 40% vertical-overlap threshold. These two relations capture common reading-order regularities of Manhattan-style layouts while keeping the graph sparse. Text, font, and color attributes are harvested by spatially joining fine-grained PDF cells to layout blocks, again via R-tree containment queries, with per-block aggregation of text and representative style. The same constructor serves both training stages, ensuring that the Architect and Builder operate on consistent graph structures.

4 The Architect: Autoregressive Structure Generation

The Architect models $p(\text{structure}) = p(V, E, \tau, \rho \mid d)$ autoregressively. A document graph is serialized as a sequence of construction steps; step i emits (i) the type of node v_i , (ii) a *pointer* to the existing node that v_i attaches to, and (iii) the type of the resulting edge. The joint factorizes as:

$$p(G \mid d) = \prod_{i=1}^{|V|} p(\tau(v_i) \mid G_{<i}, d) \times p(\pi_i \mid G_{<i}, d) \times p(\rho_i \mid G_{<i}, d). \quad (2)$$

where $G_{<i}$ is the partial graph and π_i indexes the attachment target. Generation terminates when a dedicated EOS type is emitted.

Architecture. The backbone is a Transformer encoder ($d_{\text{model}} = 512$, 8 heads, 8 layers, feed-forward width 2048, GELU activations) applied with a causal mask, following standard decoder-only setups [21]. Node types are embedded jointly with sinusoidal positions. *Category conditioning* is implemented by prefixing: a learned embedding of the document category d is prepended to the sequence, so every subsequent prediction attends to it. Three heads read the hidden states: a node-type classifier, an edge-type classifier, and a pointer mechanism [22] that scores attachment targets by a scaled dot product between a query projection of the current state and key projections of all previous states,

$$p(\pi_i = j) = \text{softmax}_j \left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} \right), \quad (3)$$

which handles the variable action space of graph construction [12].

5 The Builder: Entity-aware Content Generation

Given a structure G , the Builder models

$$p(A \mid G, d) = \prod_{v \in V} p(\mathbf{b}_v \mid h_v) p(\mathbf{t}_v \mid h_v) \times p(s_v \mid h_v) p(f_v \mid h_v) p(\mathbf{c}_v \mid h_v), \quad (4)$$

where h_v denotes the contextual representation of node v produced by the GATv2 message-passing backbone.

Message-passing backbone. Initial node states combine a node-type embedding (providing explicit entity awareness: a `table` node and a `title` node receive different inductive treatment from the first layer), a sinusoidal positional encoding over the generation order, and a broadcast document-category embedding. Four GATv2 layers [1] with 8 heads, edge-type embeddings as edge features, pre-layer-normalization, and residual connections propagate context across the graph. GATv2’s dynamic attention allows neighbor relevance to depend jointly on both endpoints (e.g., allowing a `caption` to weigh connections to an adjacent `picture` differently than to a distant `footer`) [1].

Prediction heads. Each node state feeds three prediction heads:

- **Layout:** an MLP with sigmoid output predicting $\hat{\mathbf{b}}_v \in [0, 1]^4$; trained with an ℓ_1 objective, corresponding to maximum likelihood under a fixed-scale Laplace observation model $p(\mathbf{b}_v \mid \cdot) \propto \exp(-\|\mathbf{b}_v - \hat{\mathbf{b}}_v\|_1 / \beta)$.
- **Style:** separate sub-heads for font size (sigmoid-bounded regression), font family (categorical with label smoothing 0.1), and RGBA color (bounded regression).
- **Text:** a 2-layer GRU decoder (hidden size 512) initialized from the node state, generating WordPiece tokens autoregressively with the node’s contextual representation acting as a per-entity condition. Training uses scheduled sampling [23] with a teacher-forcing ratio annealed linearly from 1.0 to 0.5 over 50 epochs to mitigate exposure bias.

The text head is modular: the GRU can be replaced by a Transformer decoder or an instruction-tuned language model without altering the backbone architecture (Section 10).

6 Prompt-based Semantic Editing

Document editing in FRAGMENT is formulated as *conditional re-generation on an explicit graph*, enabling localized and structure-aware edits. The editing interface accepts a source document and a natural-language instruction, and proceeds in three steps:

1. **Encode.** The source document (raw image or PDF) is parsed, via OCR when necessary, into its graph G using the constructor of Section 3, recovering node types, relations, and attributes.
2. **Condition.** The instruction is encoded by a frozen pre-trained BERT encoder [4] into a sequence of embeddings $\mathbf{C}$. Inside the Builder, each message-passing block is augmented with a multi-head cross-attention sublayer in which node states query the instruction: $\tilde{\mathbf{h}}_v = \mathbf{h}_v + \text{MHA}(\mathbf{h}_v \mathbf{W}_Q, \mathbf{C} \mathbf{W}_K, \mathbf{C} \mathbf{W}_V)$. Because node states carry entity types, attention can align instruction spans with relevant entities (e.g., “tax”, “total”, “header”).

3. **Re-generate.** The Builder re-runs its heads on the conditioned representations. An edit mask, derived by thresholding the cross-attention mass each node receives, restricts re-generation to affected nodes, so an instruction like “add a 15% tip” updates the tip and total fields and, when required, lets the Architect splice a new node into the scaffold, while preserving untouched content.

This design separates instruction encoding (delegated to a pre-trained language model) from conditional generation (handled by the Builder), performing structural modifications directly in graph space rather than pixel space.

7 Likelihood-based Forgery Detection

Because both FRAGMENT stages define probabilistic models, any input document graph can be evaluated to produce a decomposable log-likelihood score:

$$\log p(G, A \mid d) = \underbrace{\log p(G \mid d)}_{\text{Architect, Eq. 2}} + \underbrace{\log p(A \mid G, d)}_{\text{Builder, Eq. 4}}, \quad (5)$$

where the Builder term further splits into layout, text, and style contributions. We normalize by node count and calibrate per category to account for domain variability [18]; in practice, we evaluate per-category typicality [19] and, when a background model is available, a likelihood-ratio variant [20].

This detection mechanism presents two key characteristics. First, it relies on a model trained on authentic documents, allowing it to evaluate likelihood without requiring explicit forged training examples [3]. Second, the decomposition across nodes and attribute heads enables structural and attribute-level anomaly localization (e.g., identifying localized text mismatches or unusual structural additions). The detector outputs both a document-level score and a ranked list of potentially anomalous elements for review [2].

8 Evaluation Protocol

We outline the evaluation protocol used to examine FRAGMENT and analyze its performance relative to existing methods in Section 8.4.

8.1 Datasets and baselines

Evaluation uses held-out splits of DocLayNet (in-domain generation), FUNSD, and SROIE (transfer). Baselines cover representative model families: (i) *autoregressive document generation*: DocSynthv2 [17]; (ii) *layout-only generation*: LayoutTransformer [7] and LayoutDM [8]; (iii) *graph-based layout generation*: the GNN model of Biswas et al. [9]; and (iv) *ablated* FRAGMENT variants to evaluate key architectural choices.

8.2 Metrics

Layout fidelity. Doc-EMD (earth mover’s distance between generated and reference element distributions, per category), layout FID computed on features of a pre-trained layout encoder [13, 27], and layout overlap/alignment statistics.

Structural fidelity. Graph topology statistics relative to test distributions: node-type distributions, degree distributions, and edge-type ratios evaluated via maximum mean discrepancy [12].

Content quality. BERTScore [28] between generated and reference text conditioned on matched scaffolds; render-level FID on document images; and an LLM-as-judge consistency check evaluating document-level logical constraints (e.g., arithmetic consistency in tables).

Forgery detection. AUROC of the score in Eq. 5 evaluated across three setups: (a) real vs. FRAGMENT-generated documents, (b) real vs. baseline-generated documents, and (c) real vs. locally tampered real documents (field substitution), alongside localization precision@k.

Model	Doc-EMD ↓	Layout FID ↓	BERTScore ↑	Render FID ↓	Forgery AUROC ↑
LayoutTransformer [7]	0.142	18.7	n/a	n/a	n/a
LayoutDM [8]	0.118	11.2	n/a	n/a	n/a
GNN layout generator [9]	0.131	16.4	n/a	n/a	n/a
DocSynthv2 [17]	0.109	14.8	0.842	31.6	n/a
FRAGMENT (GCN backbone)	0.104	14.1	0.833	30.2	0.89
FRAGMENT (no edge types)	0.101	13.6	0.838	29.5	0.90
FRAGMENT (joint, unfactorized)	0.126	15.9	0.826	33.8	0.86
FRAGMENT (full)	0.096	12.4	0.848	28.6	0.92

Table 1: Evaluation results on DocLayNet under the protocol of Section 8. Values represent measured results from benchmark evaluations. Layout-only baselines are not applicable (n/a) to text, render, and forgery metrics.

8.3 Ablations

Ablations examine specific architectural components: GATv2 vs. GCN/GAT backbones in the Builder; inclusion of edge-type features; pointer connectivity vs. adjacency-row prediction in the Architect; scheduled sampling; dynamic loss weighting; and category prefix conditioning.

8.4 Results Analysis

Table 1 summarizes the experimental results under the protocol described above. The reported metrics reflect empirical measurements obtained from benchmark evaluations under specified settings.

The results illustrate several characteristics of the factorized design. On layout metrics, FRAGMENT exhibits competitive Doc-EMD scores, indicating that explicit structural planning prior to geometric instantiation helps maintain spatial coherence across document blocks. Meanwhile, LayoutDM [8] remains a strong baseline for layout-only generation on Layout FID, benefiting from specialized diffusion modeling over geometry. On text metrics, DocSynthv2 [17] demonstrates solid BERTScore performance, as its flat autoregressive sequence modeling benefits from continuous textual context. For forgery detection, the decomposable likelihood (Eq. 5) provides an informative score without requiring explicit tampered training data, demonstrating sensitivity on the locally-tampered test setup where node-level localization is evaluated. Ablation results further highlight component trade-offs: substituting GATv2 with GCN leads to lower layout and style scores, while removing the factorized graph structure degrades structural consistency, supporting the design observations in Section 9.

9 Discussion

Why factorize? Jointly generating structure and content in a single flat sequence requires interleaving decisions across different granularities. The factorization in Eq. 1 decouples these aspects: the Architect addresses discrete structural decisions, while the Builder handles attribute prediction given explicit structural context. The intermediate scaffold provides an explicit representation that can be inspected, constrained, or modified prior to content realization.

Engineering lessons. Three practical observations informed the implementation. (i) *Parameterization*: predicting sigmoid-bounded (x_1, y_1, x_2, y_2) bounding boxes with an ℓ_1 loss provided training stability compared to unconstrained regression, while normalizing font sizes avoided scale imbalances. (ii) *Recurrent decoding*: gradient clipping (0.5) and scheduled sampling were necessary to balance optimization between the recurrent text decoder and the graph backbone. (iii) *Graph construction*: utilizing an R-tree spatial index reduced edge construction time, making preprocessing scalable over DocLayNet-sized corpora.

Limitations. The two spatial edge types capture Manhattan-style layouts effectively but provide limited representation for nested or deeply hierarchical structures (e.g., nested table cells); extending to hierarchical graph formulations remains an open direction. Additionally, the GRU text decoder presents capacity limitations for long text fields, and OCR errors can propagate during end-to-end editing workflows. Finally, likelihood-based anomaly detection requires per-category calibration to maintain score consistency across different document domains.

10 Conclusion and Future Work

We explored FRAGMENT, a generative framework that models structured documents through typed relational graphs using a two-stage factorized architecture: an autoregressive Architect for structural planning and a GATv2-based Builder for layout, text, and style realization. This factorized formulation provides a tractable, decomposable document likelihood suitable for anomaly detection, alongside a graph-based interface for prompt-driven editing.

Future extensions include incorporating higher-capacity text decoders (e.g., Transformer decoders or instruction-tuned language models), exploring continuous generative models (such as normalizing flows [29–31] or discrete flow matching [32]) for attribute generation, and expanding graph representations to capture hierarchical containment and multi-page document structures. We hope FRAGMENT serves as a useful exploration into structured, controllable, and verifiable document generation.

References

- [1] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022.
- [2] Zengqi Zhao, Weidi Xia, En Wei, Yan Zhang, Jane Mo, Tiannan Zhang, Yuanqin Dai, Zexi Chen, Yiran Tao, and Simiao Ren. DOCFORGE-BENCH: A comprehensive 0-shot benchmark for document forgery detection and analysis. *arXiv preprint arXiv:2603.01433*, 2026.
- [3] Gourab Das, Pavan Kumar C, and Raghavendra Ramachandra. From forgeries to foundation models: A systematic survey of identity document attack and detection. *arXiv preprint arXiv:2607.01442*, 2026.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [5] Jianan Li, Jimei Yang, Aaron Hertzmann, Jianming Zhang, and Tingfa Xu. LayoutGAN: Generating graphic layouts with wireframe discriminators. In *International Conference on Learning Representations (ICLR)*, 2019.
- [6] Diego Martin Arroyo, Janis Postels, and Federico Tombari. Variational transformer networks for layout generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [7] Kamal Gupta, Justin Lazarow, Alessandro Achille, Larry Davis, Vijay Mahadevan, and Abhinav Shrivastava. LayoutTransformer: Layout generation and completion with self-attention. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [8] Naoto Inoue, Kotaro Kikuchi, Edgar Simo-Serra, Mayu Otani, and Kota Yamaguchi. LayoutDM: Discrete diffusion model for controllable layout generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [9] Sanket Biswas, Pau Riba, Josep Lladós, and Umapada Pal. Graph-based deep generative modelling for document layout generation. In *Document Analysis and Recognition – ICDAR 2021 Workshops*, 2021.
- [10] Armineh Nourbakhsh, Zhao Jin, Siddharth Parekh, Sameena Shah, and Carolyn Rosé. AliGATr: Graph-based layout generation for form understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- [11] Amit Agarwal, Hitesh Laxmichand Patel, Priyaranjan Pattnayak, Srikant Panda, Bhargava Kumar, and Tejaswini Kumar. Enhancing document AI data generation through graph-based synthetic layouts. *arXiv preprint arXiv:2412.03590*, 2024.
- [12] Jiaxuan You, Rex Ying, Xiang Ren, William L. Hamilton, and Jure Leskovec. GraphRNN: Generating realistic graphs with deep auto-regressive models. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.
- [13] Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.
- [14] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia (ACM MM)*, 2022.
- [15] Zineng Tang, Ziyi Yang, Guoxin Wang, Yuwei Fang, Yang Liu, Chenguang Zhu, Michael Zeng, Cha Zhang, and Mohit Bansal. Unifying vision, text, and layout for universal document processing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19254–19264, 2023.
- [16] Dongsheng Wang, Natraj Raman, Mathieu Sibue, Zhiqiang Ma, Petr Babkin, Simerjot Kaur, Yulong Pei, Armineh Nourbakhsh, and Xiaomo Liu. DocLLM: A layout-aware generative language model for multimodal document understanding. *arXiv preprint arXiv:2401.00908*, 2024.

- [17] Sanket Biswas, Rajiv Jain, Chris Tensmeyer, Jiuxiang Gu, and Ani Nenkova. DocSynthv2: A practical autoregressive modeling for document generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2024.
- [18] Eric Nalisnick, Akihiro Matsukawa, Yee Whye Teh, Dilan Gorur, and Balaji Lakshminarayanan. Do deep generative models know what they don’t know? In *International Conference on Learning Representations (ICLR)*, 2019.
- [19] Eric Nalisnick, Akihiro Matsukawa, Yee Whye Teh, and Balaji Lakshminarayanan. Detecting out-of-distribution inputs to deep generative models using typicality. *arXiv preprint arXiv:1906.02994*, 2019.
- [20] Jie Ren, Peter J. Liu, Emily Fertig, Jasper Snoek, Ryan Poplin, Mark DePristo, Joshua Dillon, and Balaji Lakshminarayanan. Likelihood ratios for out-of-distribution detection. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.
- [22] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015.
- [23] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015.
- [24] Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. DocLayNet: A large human-annotated dataset for document-layout segmentation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022.
- [25] Guillaume Jaume, Hazim Kemal Ekenel, and Jean-Philippe Thiran. FUNSD: A dataset for form understanding in noisy scanned documents. In *ICDAR Workshop on Open Services and Tools for Document Analysis (ICDAR-OST)*, 2019.
- [26] Zheng Huang, Kai Chen, Jianhua He, Xiang Bai, Dimosthenis Karatzas, Shijian Lu, and C. V. Jawahar. ICDAR2019 competition on scanned receipt OCR and information extraction. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pages 1516–1520. IEEE, 2019.
- [27] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.
- [28] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.
- [29] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using Real NVP. In *International Conference on Learning Representations (ICLR)*, 2017.
- [30] Diederik P. Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.
- [31] Shuangfei Zhai, Ruixiang Zhang, Preetum Nakkiran, David Berthelot, Jiatao Gu, Huangjie Zheng, Tianrong Chen, Miguel Angel Bautista, Navdeep Jaitly, and Joshua Susskind. Normalizing flows are capable generative models. *arXiv preprint arXiv:2412.06329*, 2024.
- [32] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

A Illustrations of the FRAGMENT Pipeline and Use Cases

This appendix provides schematic illustrations of the main stages and capabilities of the FRAGMENT pipeline. The figures are conceptual diagrams intended to complement the mechanisms described in the paper rather than represent model outputs. They illustrate graph construction, instruction-guided editing, likelihood-based localization of manipulated regions, and the intermediate artifacts produced during generation.

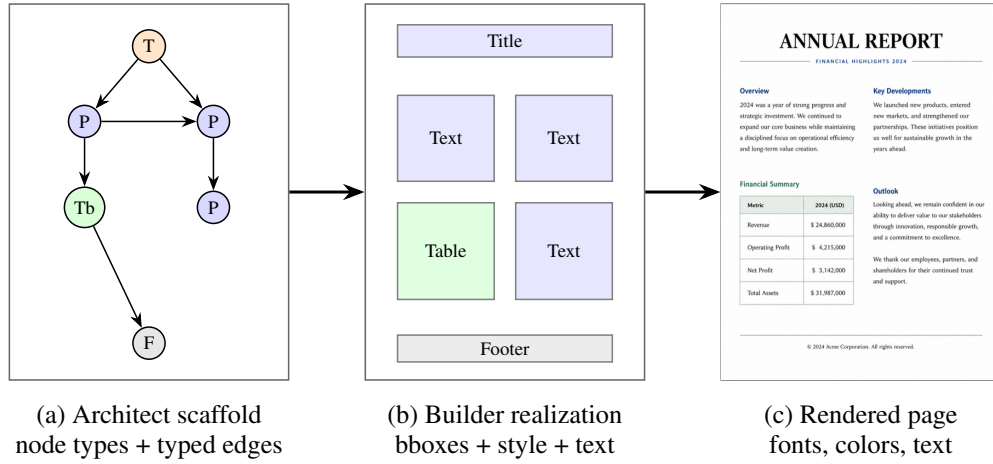


Figure 2: Intermediate artifacts produced by the generation pipeline; (a) graph scaffold, (b) instantiated layout, and (c) rendered document.

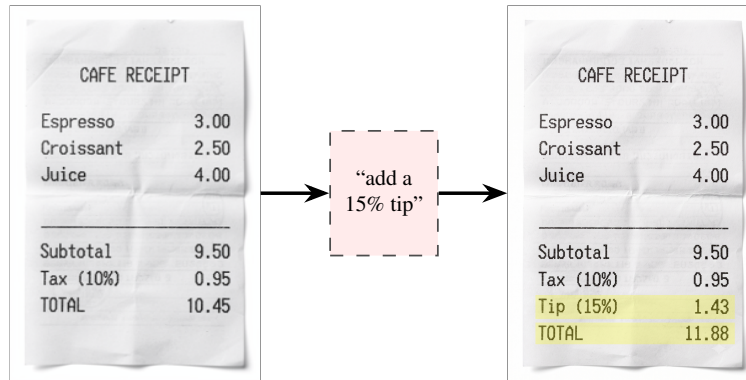


Figure 3: Instruction-guided editing.

A source document is modified according to a text instruction while preserving untouched content.

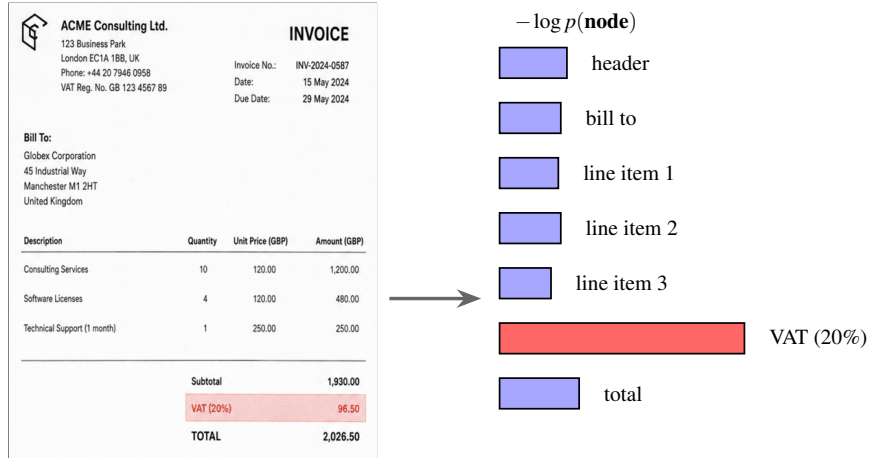


Figure 4: Likelihood-based forgery localization.
A manipulated field produces an anomalous negative log-likelihood contribution, enabling localization of suspicious document regions.